

AI problem solving through searching

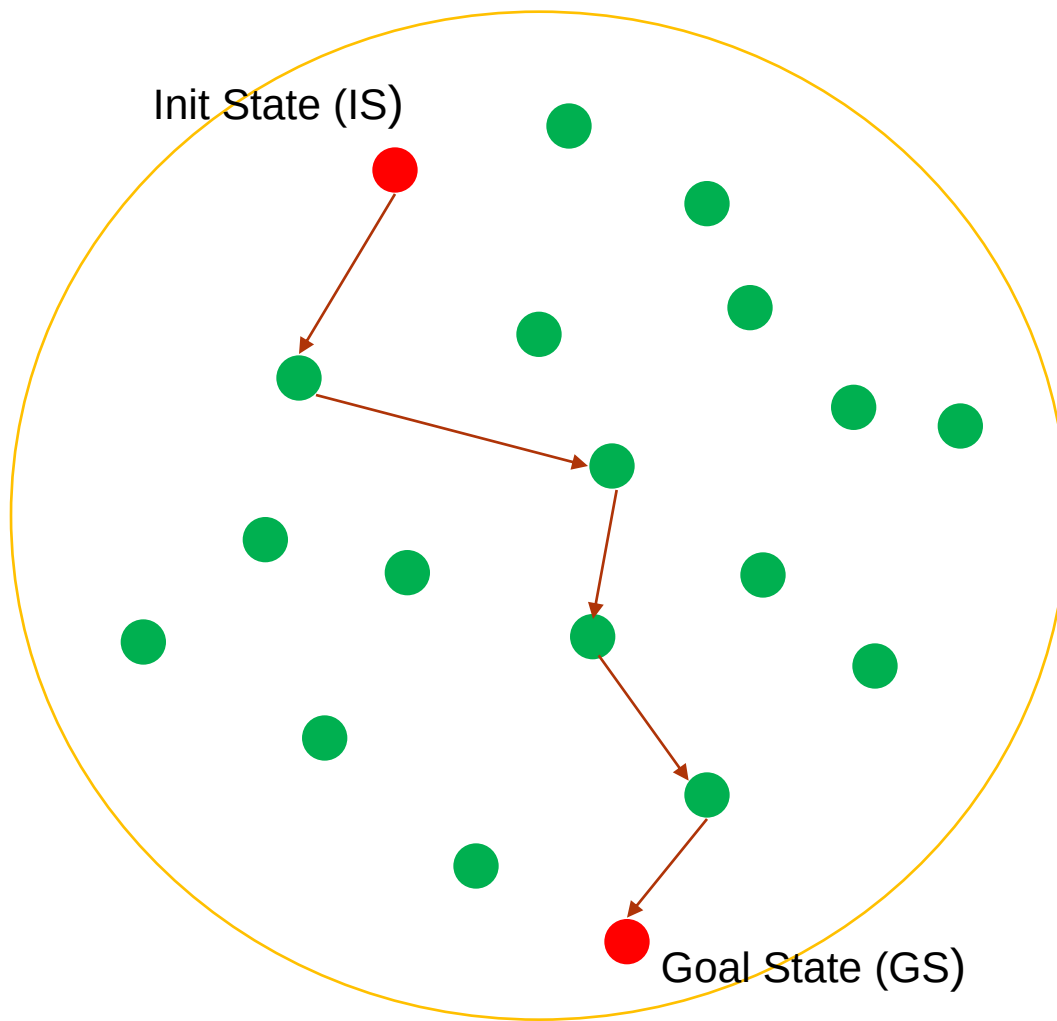
Dr. Manas Ranjan Patra

AI problem solving techniques:

1. **Problem Solving by Searching**
(tick-tac-toe, 8-puzzle, chess)
2. **Problem Solving by Reasoning/ Inferencing**
(Diagnosis, fault-finding)
3. **Problem Solving by Matching**
(face recognition, content similarity)

Defining an AI Problem as **State Space Search**:

1. Define a **State Space** containing all possible configurations (states) of the relevant objects associated with the problem.
2. Specify the **Initial State** within the state space from which problem solving may begin.
3. Specify the **Goal State** which represents an acceptable solution (where the search will terminate)
4. Specify a **set of rules** that describe permissible actions.



State Space vs Search Space

State Space:

All possible states that may arise for the problem under consideration.

Ex: In chess game, the state space would include all possible board configurations/situations (wherever a piece can be placed) during the game.

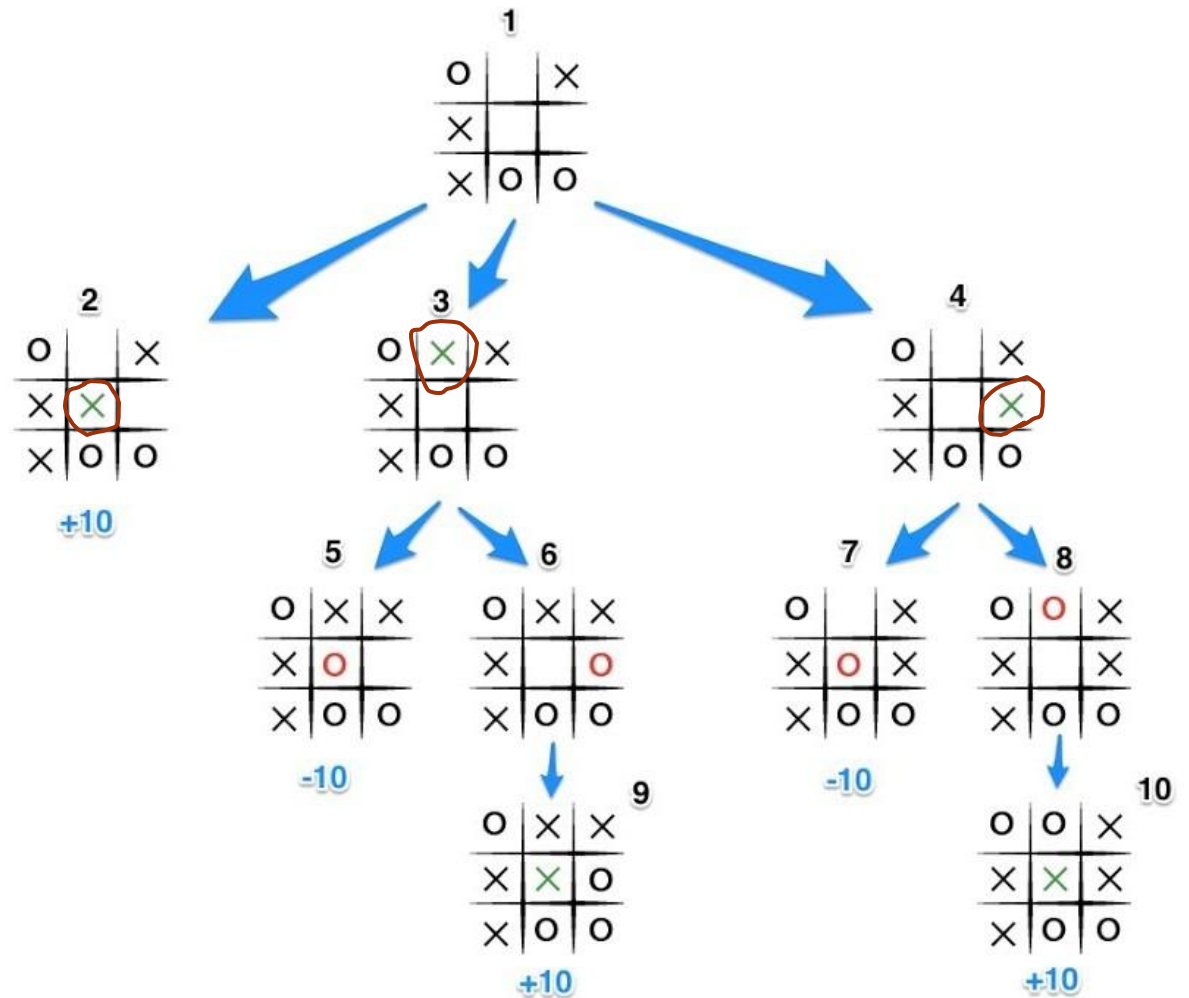
Search Space:

It is a subset of the state space that is actually explored/navigated during the problem-solving process while trying to find a solution.

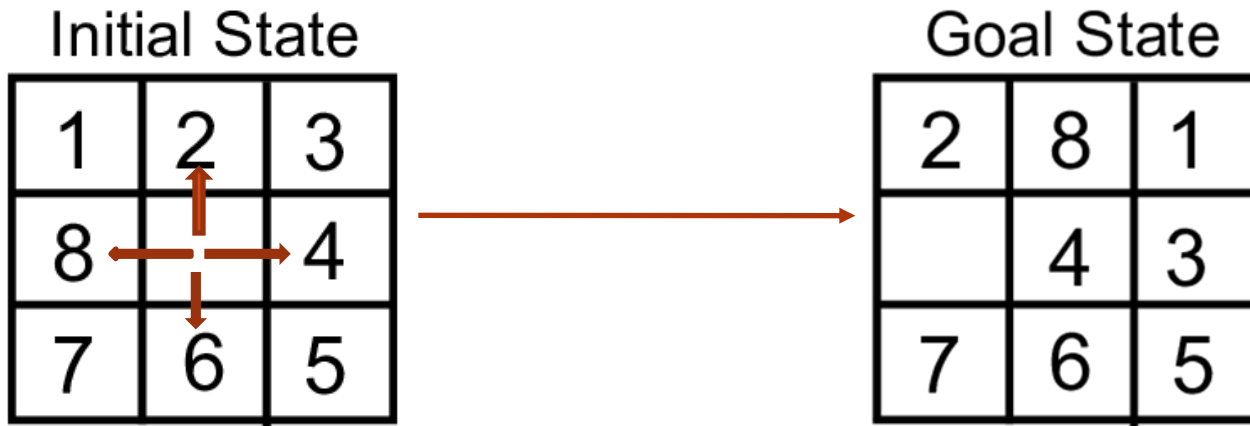
Ex: Only those board configurations/states that are examined during the search for a solution (the best move).

Tic Tac Toe

O	X	
O	X	O
X	O	X



8 Puzzle problem



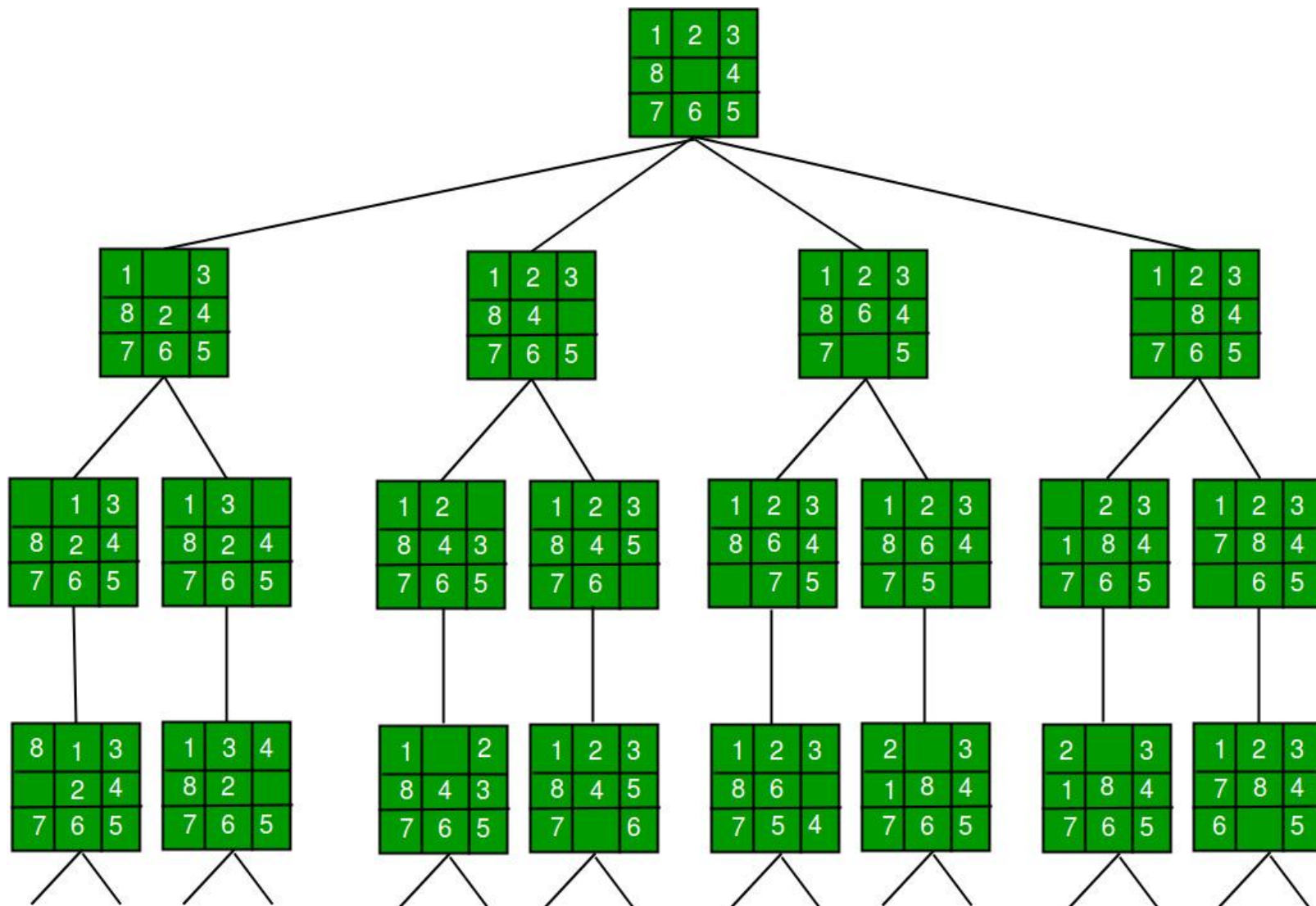
Operators:

Move UP

Move Down

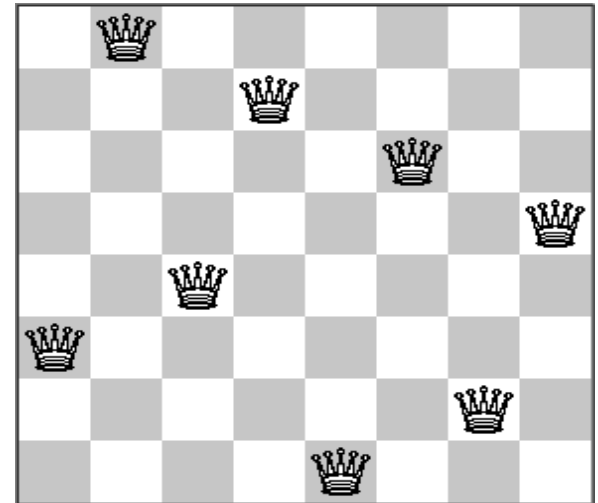
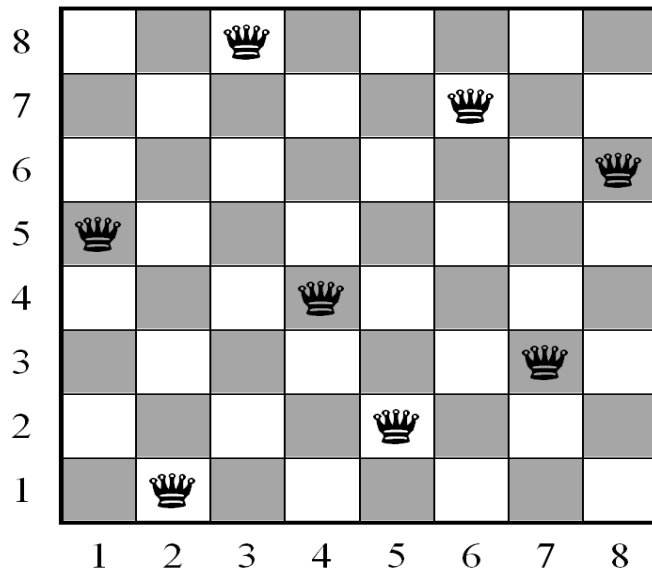
Move Left

Move Right



8 Queen problem

The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other queen on the chessboard. (A queen attacks any piece in the same row, column or diagonal.)



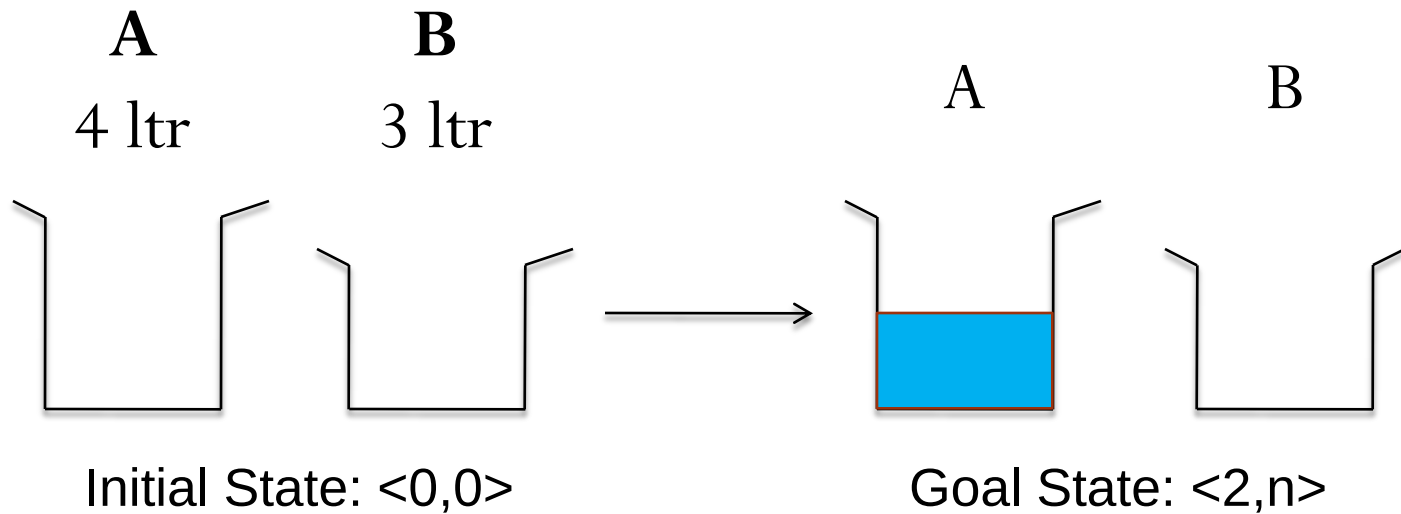
Control Strategy

- Control Strategy in AI refers to a strategy that helps one to select a rule that can be applied at every step while searching for a desired goal state in a state space.
- The objective is to design an efficient control strategy to guide the search process in a state space for finding a solution.

Characteristics of a good control strategy

- A control strategy must cause motion (move from one state to another) otherwise a soln. can never be reached.
- A control strategy must be systematic.
This emphasizes on Local motion & Global motion

A Water Jug Problem



State Space: Described as $\langle x, y \rangle$ ordered pair

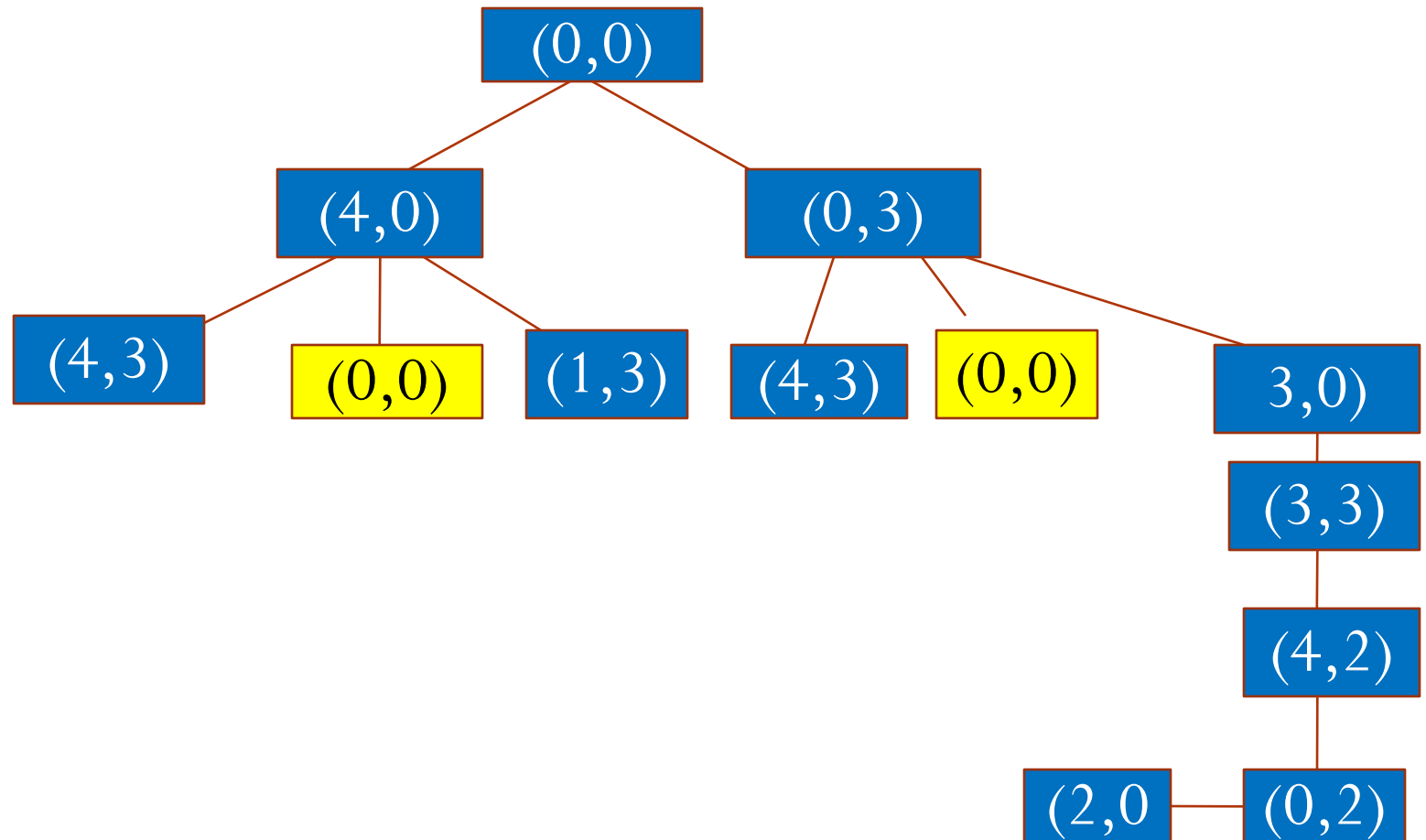
$X = \{0, 1, 2, 3, 4\}$

$Y = \{0, 1, 2, 3\}$

Modeling Water jug problem as a state space search problem

- Construct a tree with the Initial state(IS) as its Root.
- Generate all possible child nodes of the root by applying appropriate rules to the IS.
- For each child node generate further child nodes by applying suitable rules at that stage.
- Continue the process till the goal state is reached.

State Space for the Water jug problem

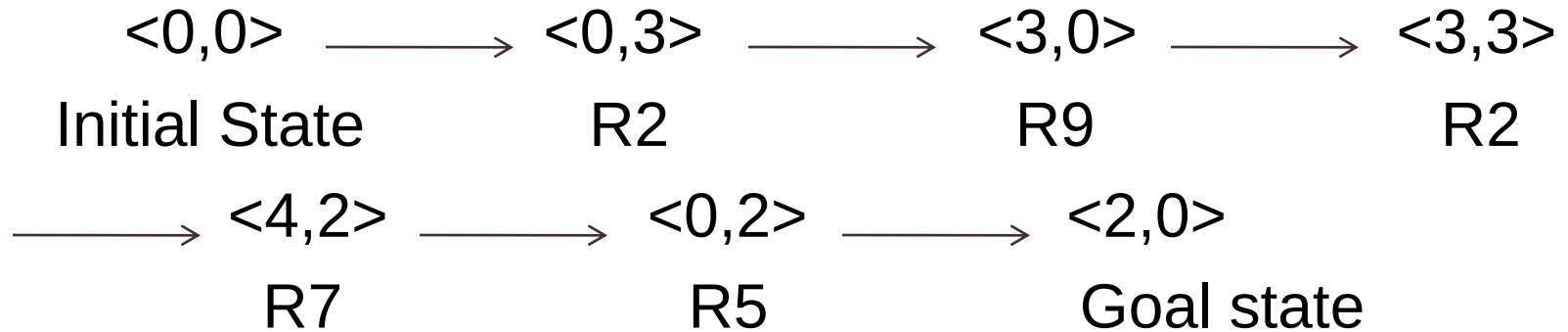


Creating a rule base:

	<u>L.H.S of rule</u>	<u>R.H.S value</u>	<u>Action</u>
R1:	$\langle x, y \rangle$ if $x < 4$	$\longrightarrow \langle 4, y \rangle$	Fill A
R2:	$\langle x, y \rangle$ if $y < 3$	$\longrightarrow \langle x, 3 \rangle$	Fill B
R3:	$\langle x, y \rangle$ if $x > 0$	$\longrightarrow \langle x - d, y \rangle$	Pour some water out of A
R4:	$\langle x, y \rangle$ if $y > 0$	$\longrightarrow \langle x, y - d \rangle$	Pour some water out of B
R5:	$\langle x, y \rangle$ if $x > 0$	$\longrightarrow \langle 0, y \rangle$	Empty A on ground
R6:	$\langle x, y \rangle$ if $y > 0$	$\longrightarrow \langle x, 0 \rangle$	Empty B on ground

	<u>L.H.S of rule</u>	<u>R.H.S value</u>	<u>Action</u>
R7:	$\langle x, y \rangle$ if $x+y \geq 4$ & $y > 0$	$\longrightarrow \langle 4, y-(4-x) \rangle$	Pour from B into A until A is full
R8:	$\langle x, y \rangle$ if $x+y \geq 3$ & $x > 0$	$\longrightarrow \langle x, (3-y), 3 \rangle$	Pour from A into B until B is full
R9:	$\langle x, y \rangle$ if $x+y \leq 4$ & $y > 0$	$\longrightarrow \langle x+y, 0 \rangle$	Pour all from B into A
R10:	$\langle x, y \rangle$ if $x+y \leq 3$ & $x > 0$	$\longrightarrow \langle 0, x+y \rangle$	Pour all from A into B
R11:	$\langle 0, 2 \rangle$	$\longrightarrow \langle 2, 0 \rangle$	Pour 2lts from B to A
R12:	$\langle 2, y \rangle$	$\longrightarrow \langle 0, y \rangle$	Pour 2lts in A on the ground

Possible Rule Sequence:



Problem Solving Strategy:

1. Define problem's state space including initial & goal states.
2. Define a set of valid operators to move in the state space.
3. Search for a path from Initial state to Goal state.

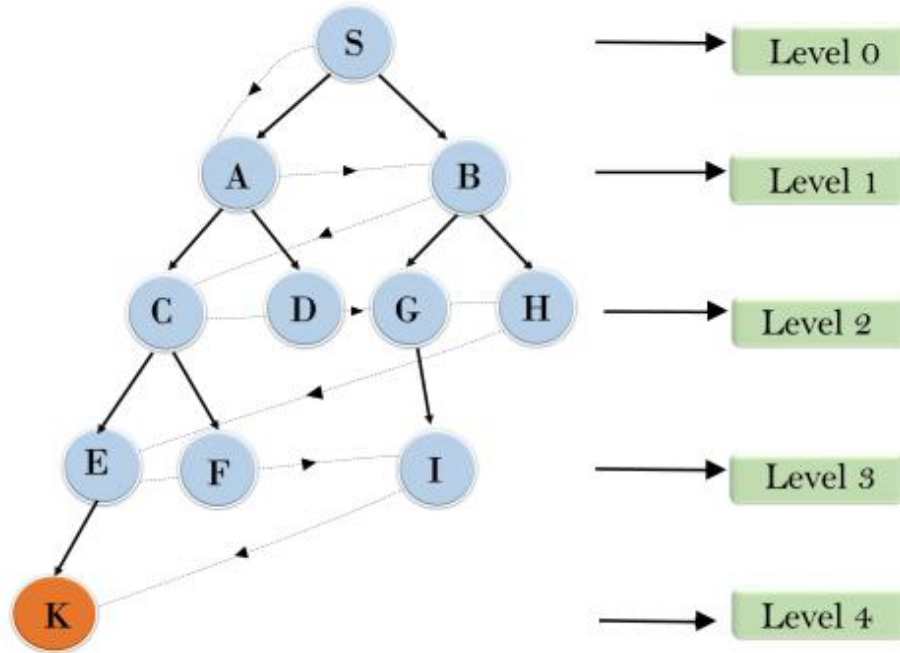
“Search” forms the core of solving AI Problems.

“Effort is to search quickly”.

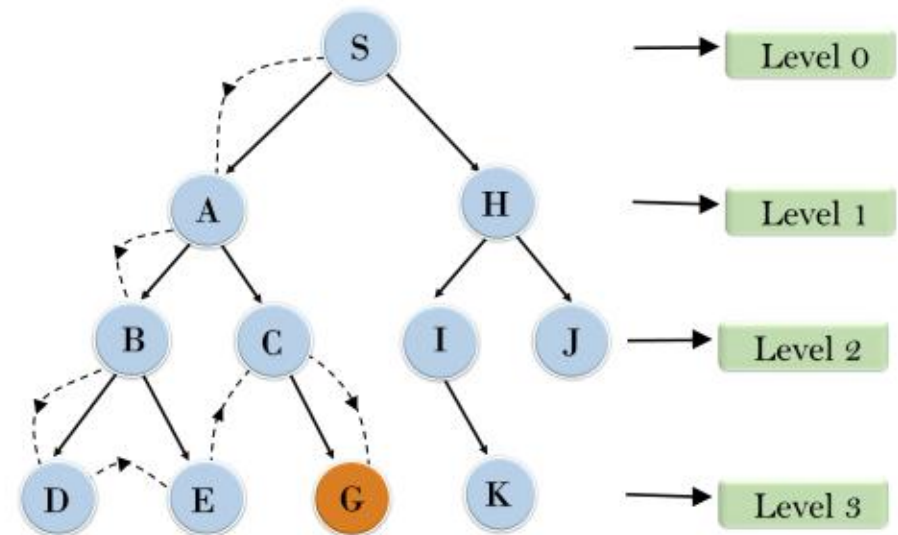
Search techniques

- **Uninformed Search (Blind search):** Here, the search space is explored without any additional information about the search space.
 - Breadth First Search (BFS)
 - Depth First Search (DFS)
 - Depth-Limited Search (DLS)
 - Iterative Deepening Search (IDS)
 - Iterative Broadening Search (IBS)
- **Informed Search (Heuristic Search):** Additional information is available to navigate in the state space.
 - Hill climbing
 - Best First Search, A*
 - Problem Reduction, AO*
 - Simulated Annealing

Breadth First Search



Depth First Search



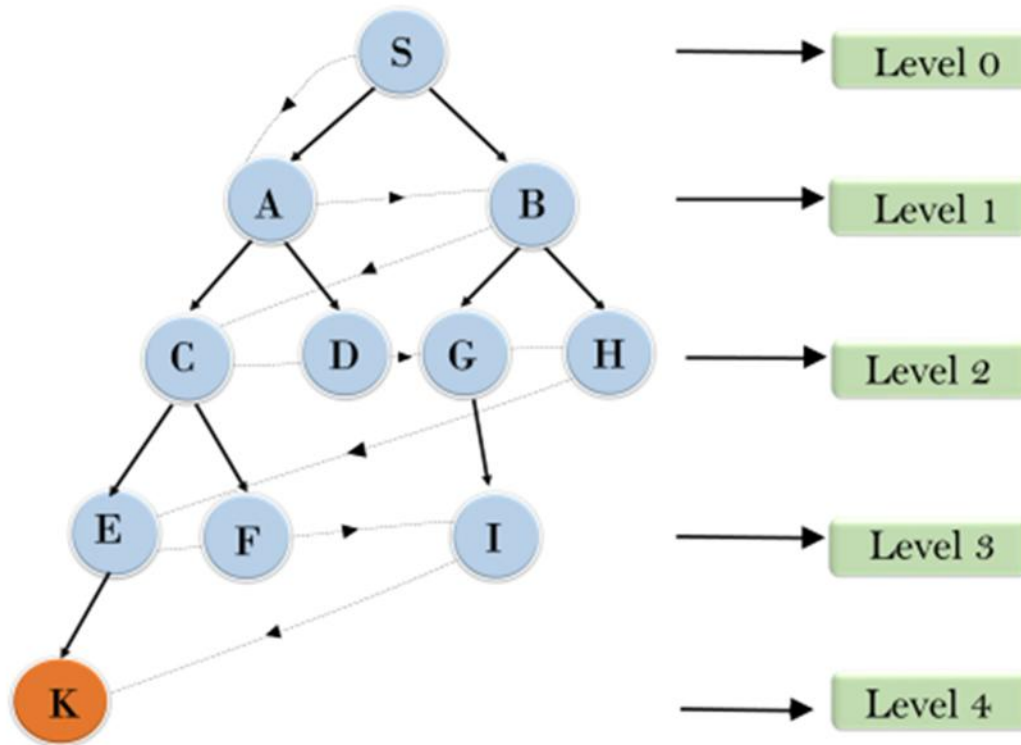
Breadth First Search (BFS)

- **Breadth-first search** is a strategy in which the root node is expanded first, then all the successors of the root node are expanded, then *their* successors, and so on i.e., all the nodes at a given depth are expanded first before going for the next level.
- BFS can give the shallowest path to a goal node.
- BFS uses a FIFO queue for its implementation

Algorithm: BFS

1. Insert the **Initial State** into a Queue data structure, called NODE-LIST.
2. Until a **Goal State** is reached or Queue is empty **do**:
 - i) **If** NODE-LIST is empty then quit
else remove the front item from NODE-LIST & call it V (Visited).
 - ii) For each rule (from the rule set) where L.H.S matches with the state described in V **do**:
 - a) Apply the rule to generate a **New State**.
 - b) If the **New State** is a **Goal State** then quit & return this state.
 - c) Otherwise, add the **New State** to the end of NODE-LIST.

Breadth First Search



Let's consider a uniform tree where every state has **b successors**. The root of the search tree generates **b** nodes at the 1st level, each of which generates **b** more nodes (i.e., b^2 nodes at level 2), and so on. If the goal state is at **depth d** & in the worst case, it is the last node generated at that level. Then the total number of nodes generated is

- **Time Complexity:**

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

$O(b^d)$: when the goal node is the 1st one at level d (leftmost)

$O(b^{d+1})$: when the goal node is the last one at level d (rightmost)

- **Space Complexity: $O(b^d)$**

where **b** is the branching factor and **d** is the solution depth

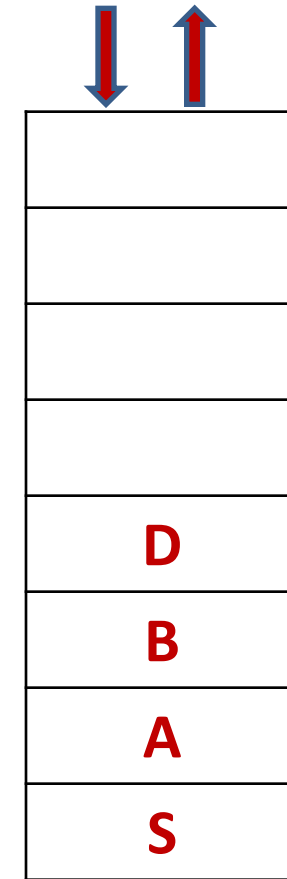
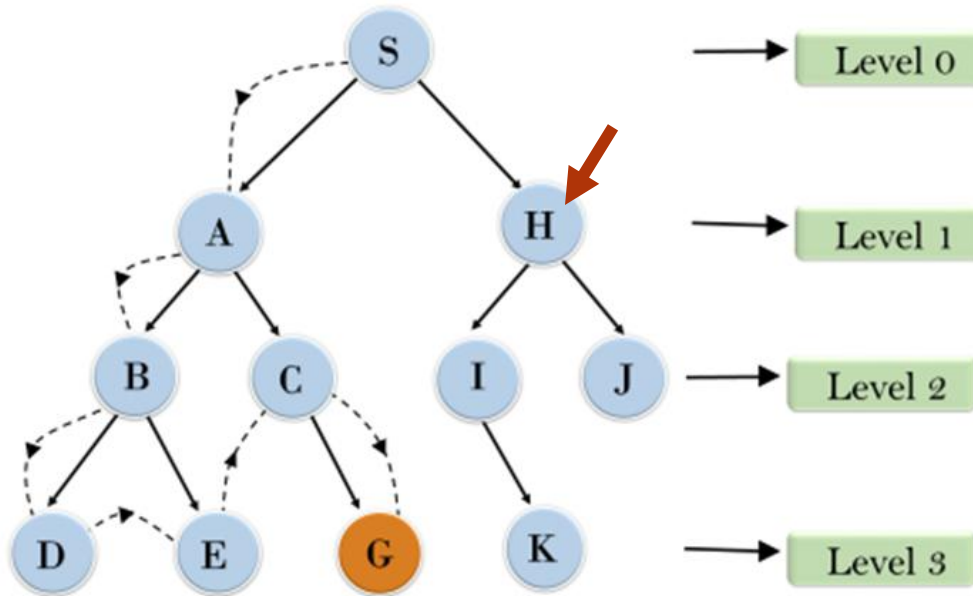
Depth First Search (DFS)

- DFS always tries to explore the deepest level until no successor is found. Then it backs up to the next deepest node that still has unexplored successors.
- It is implemented as a recursive function.
- It uses a Stack (LIFO list) for its implementation.

Algorithm DFS:

1. If initial state is **Goal-state**, Quit & return success.
2. Otherwise, repeat the following until success or failure is indicated.
 - i) Generate a successor **V** of the **Init-state**.
If no more successor, indicate failure.
 - ii) Call DFS with **V** as the **Init-state**.
 - iii) If **Goal-state** is reached, indicate success else continue in this loop.

Depth First Search



Time Complexity:

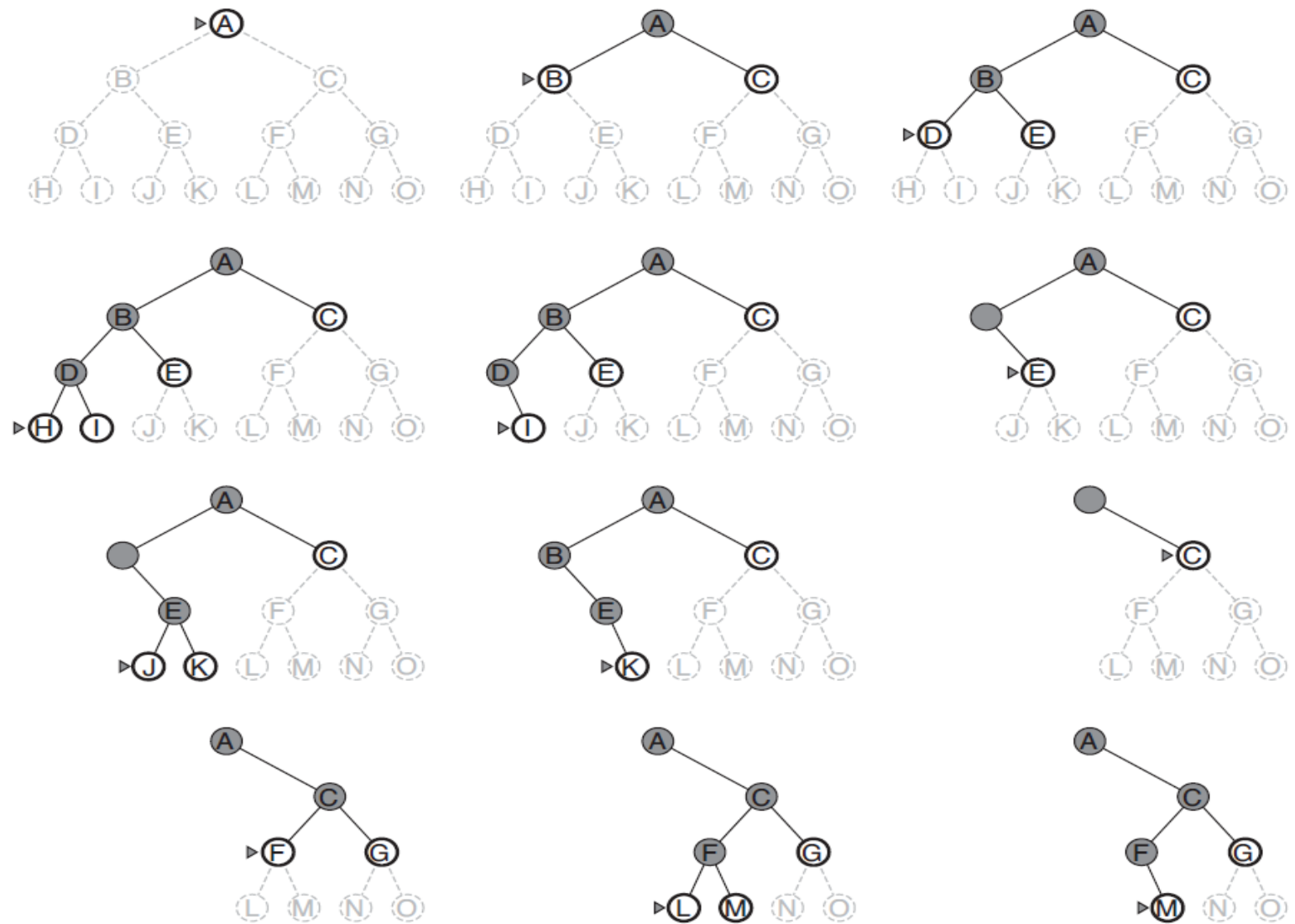
$$O(b^m)$$

Space Complexity:

$$O(bm)$$

where b is branching factor & m is the maximum depth of search tree

DFS needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes for each node along the path.



Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and *M* is the only goal node.

Back tracking Search

Strategy:

Proceed along a single path until a **Goal-state** is reached or decision to terminate is taken.

When to terminate?

- when a Dead-end is reached.
- when a previous state is produced.
- when Path becomes longer than some predefined “Futility Limit”.

“Chronological Back tracking”

Backtrack to a most recently created state (parent node) where other alternative moves were available. Select one of the alternatives & repeat the process.

- In backtracking, only one successor is generated at a time rather than all successors.
- Therefore, only $O(m)$ memory is needed rather than $O(bm)$.

Depth-Limited Search (DLS)

- DLS is a variation of DFS that sets a limit on how deep the search can go.
- It is suitable when the search space is extremely large and the tree being searched is very deep.
- DFS may get stuck in case of very deep search spaces as it may never find a solution.
- In DLS, a maximum depth limit is set, and the algorithm will not explore beyond this limit. If a node at the maximum depth is reached without finding a solution, the algorithm has to backtrack.

Process

1. Start at the root node (initial state)
2. Call DFS recursively
3. Stop exploring a path further once the specified depth limit is reached
4. If the goal state is found within this depth, then return the Goal state
5. Else backtrack and continue exploring other branches within the depth limit.

Recursive implementation of DLS

function DEPTH-LIMITED-SEARCH(*problem*, *limit*) **returns** a solution, or failure/cutoff
 return RECURSIVE-DLS(MAKE-NODE(*problem*.INITIAL-STATE), *problem*, *limit*)

function RECURSIVE-DLS(*node*, *problem*, *limit*) **returns** a solution, or failure/cutoff
 if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)
 else if *limit* = 0 **then return** *cutoff*
 else
 cutoff_occurred? $\leftarrow$ false
 for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**
 child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)
 result $\leftarrow$ RECURSIVE-DLS(*child*, *problem*, *limit* - 1)
 if *result* = *cutoff* **then** *cutoff_occurred?* $\leftarrow$ true
 else if *result* $\neq$ *failure* **then return** *result*
 if *cutoff_occurred?* **then return** *cutoff* **else return** *failure*

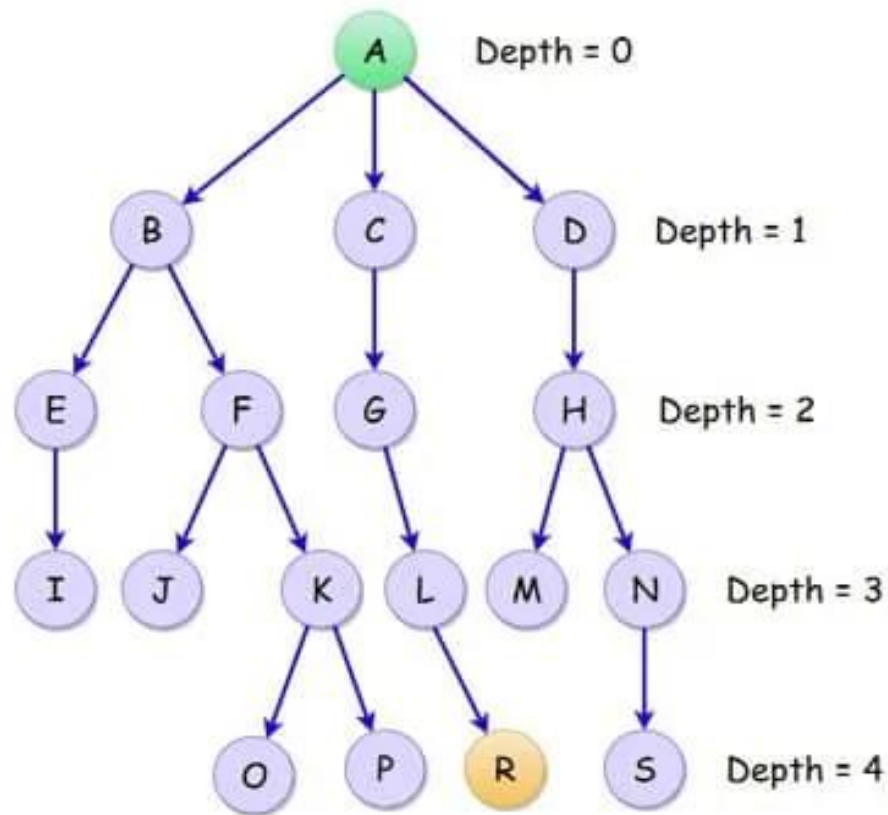
Iterative Deepening Search (IDS)

- It combines DFS's space-efficiency and BFS's fast search.
- DFS and BFS have limitations. DFS may go too deep quickly, while BFS might explore too broadly and consume a lot of memory. Iterative Deepening Search (IDS) is an alternative that addresses some of these limitations by progressively deepening the search level.
- Iterative Deepening Search (IDS) (also called, iterative deepening depth-first search-IDDFS) is a state space search strategy in which a depth-limited version of depth-first search is executed repeatedly with increasing depth limits until the goal is found.

Process

IDS performs a series of depth-limited searches, starting with a depth limit of 0 and increasing this limit by one with each iteration

1. Set depth limit, $d = 0$
2. Check if the start node is the goal state
3. Increment depth, $d = d + 1$
4. Perform a DLS upto depth d
5. If goal is found, return solution
6. Else backtrack and repeat the process within the depth limit d
7. Repeat the process by incrementing d in each iteration until a goal state is found or a predefined maximum depth is reached.



DEPTH LIMITS

0

1

2

3

4

IDDFS

A

A B C D

A B E F C G D H

A B E I F J K C G L D H M N

A B E I F J K O P C G L R D H M N S

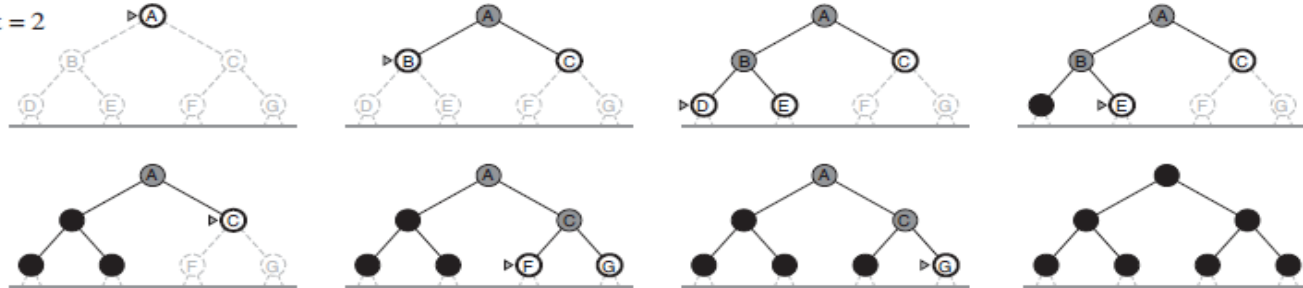
Limit = 0



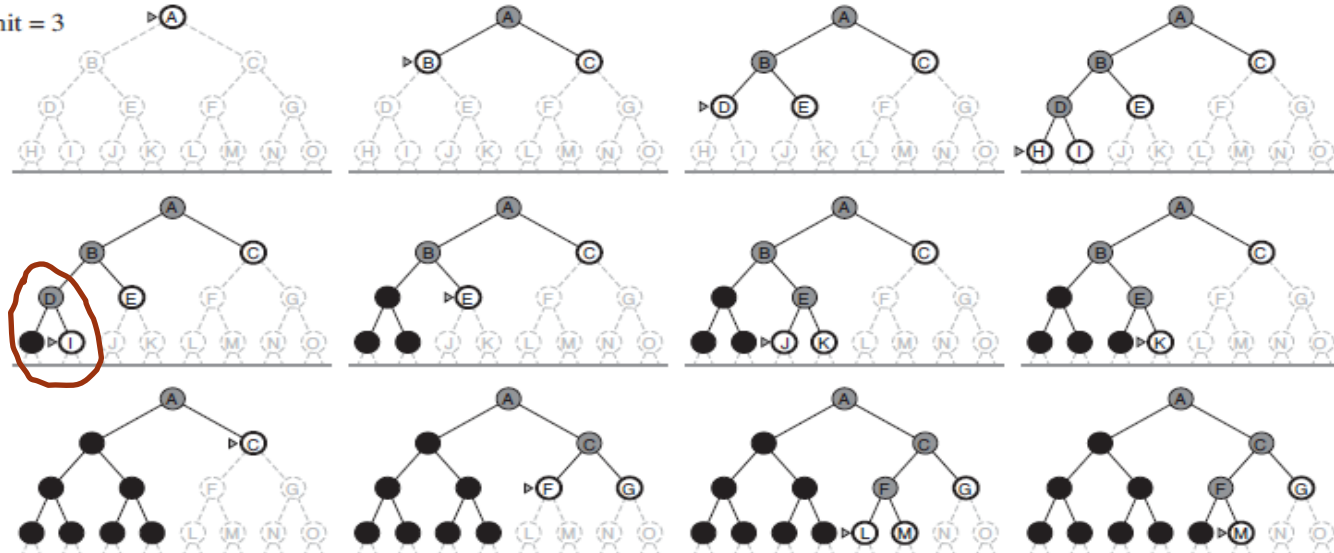
Limit = 1



Limit = 2



Limit = 3



Four iterations of iterative deepening search on a binary tree.

Algorithm: IDS

function IDS(root)

for depth from 0 to (some limit) do

found, remaining $\leftarrow$ **DLS(root, depth)**

// provide the depth within which
solution is to be found

if (found $\neq$ null) then return found

else if not remaining then return null

Contd..

function DLS(node, depth) // in each iteration IDS calls DLS

if depth = 0 then

if node is a goal then //Goal is at depth-0

return (**node**)

else

return (**null**) //Goal Node not found at depth 0

else if depth > 0 then // move to next level

while (remaining == true)

for each child of node do

found, remaining ← **DLS(child, depth-1)** //recursive call

if found ≠ null then

return (found) //Goal node is found at a level

if remaining then

remaining ← true //Go to the next level

return

Iterative Broadening Search (IBS)

- While IDS controls the depth, IBS controls the breadth of the search.

Principle:

1. The algorithm begins by exploring a limited number of branches at the 1st level.
2. After exploring these branches, it restarts the search with a broader scope, increasing the number of branches explored at each level.
3. The process is repeated iteratively, with the breadth of search being progressively expanded until the solution is found or a certain criterion is met.

function IBS(problem):

for breadth_limit in range(1, MAX_BREADTH)

result = search_with_limited_breadth (problem, breadth_limit)

if result is found:

return result

else return failure

function search_with_limited_breadth(problem, breadth_limit):

Implement a search strategy (e.g., depth-first search)

but limit the number of branches explored at each level
to breadth_limit

...

Advantages

- **Memory Efficiency:** Like IDS, IBS is memory-efficient compared to full breadth-first search because it doesn't attempt to explore all branches at once.
- **Balanced Exploration:** It strikes a balance between exploring a broad range of possibilities and going deeper into the most promising paths.
- **Applicability** : Useful in scenarios like game tree exploration (Chess) where the number of possible moves (branching factor) can be large and it's difficult to explore all possible options at each step.